



CODING GUIDE

Edit

Sketch Overview

Almost all Arduino Sketches share a common structure, consisting of a `setup` method and a `loop` method. This structure remains unchanged when integrating with Thinger.io. However, it is important to understand where device resources should be defined and where interaction with external services is possible. In general terms, any device resource (such as an LED, relay, sensor, or servo) must be defined inside the `setup()` method. Similar to initializing devices, setting the input/output direction of a digital pin, or initializing the Serial port speed, resources also need to be initialized here. This essentially involves configuring which values or resources are to be exposed over the Internet.

The `loop()` is the designated place to consistently call the `thing.handle()` method, allowing the Thinger libraries to manage the platform connection. This method also serves as the location for calling endpoints or streaming real-time data to a dashboard. It is important to avoid adding any delays within the `loop()` unless specific actions, such as working with deep sleep modes on a device, are being implemented. Any other delay will negatively impact Thinger's proper functioning on the device. Additionally, reading a sensor value in every loop iteration can be detrimental if the sensor requires significant time to complete a read, as this will lead to a device with noticeable lag when responding to commands.

```
// add required headers according to the device
#include <ThingerESP32.h>

// initialize Thinger instance (type can change depending on the
device)
ThingerESP32 thing("username", "deviceId", "deviceCredential");

void setup() {
    // initialize sensors and pins

    // initialize wifi (see examples for the device)

    // add resources here, like sensors, lights, etc.
}

void loop() {
    // call always the thing handled in the loop and avoid any delay
here
    thing.handle();
    // here it is possible to call endpoints
    // and also it is possible to stream resources
}
```

Setting Credentials

All devices connected to the platform require authentication against the server. When a device is created in the console ↗, a new device identifier is generated and device credentials are set. Therefore, these credentials must also be configured in the Arduino code to allow the device to be recognized and associated with the account. This is typically done during the initialization of the Thinger instance in the code, specifically when the `thing` instance is defined. The `username`, `deviceId`, and `deviceCredential` should be replaced with the values registered in the cloud. It is worth noting that credentials used to be defined inside `arduino_secrets.h`.

```
ThingerESP32 thing("username", "deviceId", "deviceCredential");
```

Adding Resources

In the Thingier.io platform, each device can define several resources. A resource can be considered anything that can be sensed or activated. For example, typical resources include a sensor value like temperature or humidity, or a relay that controls a light. Therefore, the resources that need to be exposed over the Internet should be defined.

All resources must be defined inside the `setup()` method of the Arduino sketch. This way, the resources are configured at the beginning, but can be accessed later as necessary.

There are three different types of resources, which are explained in the following sections.

Input Resources

If control or actuation of an IoT device is required, an input resource must be defined. An input resource serves as a source of information for the device. Examples include resources for controlling a light or relay, adjusting a servo position, or modifying a device parameter.

To control or actuate an IoT device, it is necessary to define an input resource. An input resource is anything that can provide information to a device. Examples include a resource for turning a light or a relay on and off, changing a servo position, or adjusting a device parameter.

To define an input resource the operator `<<` is used, pointing to the resource name, and it uses a C++11 Lambda function to define the function.

The input resource function takes one parameter of type `pson` that is a variable type that can contain booleans, numbers, floats, strings, or even structured information like in a JSON document.

The following subsections will show how to define different input resources for typical use cases.

Turn on/off a LED, a relay, etc

This kind of resource only requires an on/off state, so it can be enabled or disabled as required. As the `pson` type can hold multiple data types, we can think that the `pson` parameter of the input function is like a boolean.

So, inside the `setup` function, place a resource called `led` (but use any other name), of input type (using the operator `<<`), that takes a reference to a `pson` parameter. This example will turn on/off the digital pin 10 using a ternary operator over the `in` parameter.

```
thing["led"] << [](pson& in){
    digitalWrite(10, in ? HIGH : LOW);
};
```

Modify a servo position

Modifying a servo position is quite similar to turning on/off a LED. In this case, however, it is necessary to use an integer value. As the `pson` type can hold multiple data types, we can still use the `pson` type as an integer value.

```
thing["servo"] << [](pson& in){
    myServo.write(in);
};
```

Update sketch variables

Input resources can also be used to update sketch variables, allowing for dynamic changes in device behavior. This is quite useful in situations where it is desirable to temporarily disable an alarm, change reporting intervals, update a hysteresis value, and so on. In this way, additional resources can be defined to change variables.

```
float hysteresis = 0; // defined as a global variable
thing["hysteresis"] << [](pson& in){
    hysteresis = in;
};
```

Pass multiple data

The `pson` data type can hold not only different data types, but also is fully compatible with JSON documents. The PSON data type can be utilized to receive multiple values simultaneously. This example will receive two different floats that are stored with the `lat` and `lon` keys.

```
thing["location"] << [](pson& in){
    float lat = in["lat"];
    float lon = in["lon"];
};
```

Show Input Resources State in Dashboards and API

The Dashboards or API work in a way that when opening them, they query the associated resources to correctly print their current state, i.e., the switch is on or off. In this way, when the API or a Dashboard is open, each associated input resource is called, receiving empty data in the call, as there is no intention to control the resource (the pson input will be empty).

So, how do the Dashboards or the API know what is the current state of an input resource? The resource must set its current state in the input parameter, if it is empty, or use the input value if there is one. This way, we can obtain three different things: query the current resource state (without modifying it), modify the current resource state, and obtain the expected input on the resource (this is how the API explorer on the device works).

Therefore, a correct input resource definition that actually allows to display of the current state of the resource in a Dashboard or in the API, will be like this example code.

```
thing["resource"] << [](pson& in){
    if(in.is_empty()){
        in = currentState;
    }
    else{
        currentState = in;
    }
};
```

This sample code basically returns the current state (like a boolean, a number, etc) if there is no input control, or uses the incoming data to update the current state. This can be easily adapted for controlling a LED, while showing its current state in the dashboard once opened or updated.

```
thing["led"] << [](pson& in){
    if(in.is_empty()){
        in = (bool) digitalRead(pin);
    }
    else{
        digitalWrite(pin, in ? HIGH : LOW);
    }
};
```

Note: For controlling a digital pin, just use the method explained in the Easier Resources Section.

Output Resources

Output resources should be used in general when needed to sense or read a sensor value, like temperature, humidity, etc. So the output resources are quite useful for extracting information from the device.

To define an output resource it is used the operator `>>` to point to the resource name, and it uses a C++11 Lambda function to define the output function.

The output resource function takes one parameter of `pson` type that is a variable type that can contain booleans, numbers, floats, strings, or even structured information like in a JSON document.

The following subsections will show how to define different output resources for typical use cases.

Read a sensor value

Defining an output resource is quite similar to defining an input resource, but in this case it is used the operator `>>`. In the callback function, we can fill the output value with any value we want, like in this case, the output from a sensor reading.

```
thing["temperature"] >> [](pson& out){
    out = dht.readTemperature();
};
```

Read multiple datasets

In the same way, the input resources can receive multiple values at the same time, the output resources can also provide multiple data. This is an example of providing both latitude and longitude from a GPS.

```
thing["location"] >> [](pson& out){
    out["lat"] = gps.getLatitude();
    out["lon"] = gps.getLongitude();
};
```

Read sketch variables

If the sketch cannot provide a single sensor reading, as it is doing some kind of data integration, an output resource can also be used for reading the sketch variables, where the computed result is updated frequently.

```
float yaw = 0; // defined as a global variable
thing["yaw"] >> [](pson& out){
    out = yaw;
};
```

Input/Output Resources

The last resource type is a resource that not only takes an input or an output, but takes both parameters. This is particularly useful when an output is dependent on an input, such as when a changing reference value needs to be provided to a sensor.

These kinds of resources are defined with the operator `=`. In this case, the function takes two different `pson` parameters. One for input data and another for output data. This example provides an altitude reading using the BMP180 Sensor. It takes the reference altitude as input and provides the current altitude as output.

```
thing["altitude"] = [](pson& in, pson& out){  
    out = bmp.readAltitude(in);  
};
```

Also, define more complex input/output resources that take several input values, to provide multiple output values, like in this example that takes `value1` and `value2` to provide the `sum` and `mult` values.

```
thing["in_out"] = [](pson& in, pson& out){  
    out["sum"] = (long)in["value1"] + (long)in["value2"];  
    out["mult"] = (long)in["value1"] * (long)in["value2"];  
};
```

Resources without parameters

It is also possible to define resources that do not require any input or generate any output. These are like callbacks that can be executed as needed, for example, to reboot the device or perform a required action.

In this case, the resource is defined as a function without any input or output parameters.

```
thing["resource"] = [](){  
    // write here the execution code  
};
```

Easier Resources

The client library also includes some useful syntactic sugar definitions for declaring resources more easily without having to think about input or output resources. These syntactic sugar features are macros that are expanded automatically to define the resources in the standard way.

The advantage of using this kind of definition is that resources will be able to handle the state when queried from the API. For example, if a digital pin is enabled or disabled, its current state will be visible in both the API explorer and a dashboard.

Control a digital pin

This kind of resource will allow defining a resource for declaring control over a digital pin, so it is possible to alternate over on/off states, which can be used for controlling a LED, a relay, a light, etc.

It is required to define the digital pin as OUTPUT in the setup code, or the resource will not work properly.

```
thing["relay"] << digitalPin(PIN_NUMBER);  
thing["relay"] << invertedDigitalPin(PIN_NUMBER);
```

Define Output Resources

This kind of resource will allow defining a resource for declaring a read-only resource, like a value obtained from a sensor, or a given variable in our sketch.

In this example, we are defining a resource that exposes a sensor reading, like the DHT11 sensor temperature.

```
thing["temperature"] >> outputValue(dht.readTemperature());
```

But it is also possible to define an output resource for any global variable in our sketch.

```
thing["variable"] >> outputValue(myVar);
```

Modify Sketch Variables

Our sketch usually defines some parameters or variables that are used inside the loop code. These kinds of resources are normally used to handle or control the execution behaviour. With these kinds of resources, we can modify any parameter we want to expose, like a float, an integer, a boolean, etc.

In this example, it is possible to remotely modify the boolean `sdLogging` variable defined as a global variable.

```
thing["logging"] << inputValue(sdLogging);
```

It is also possible to define a callback function to know when the variable has changed, so we can perform any other action. For this use case, define the resource to have some code executed when the `hysteresisVar` changes.

```
thing["hysteresis"] << inputValue(hysteresisVar, {  
  // execute some code when the value changes  
  Serial.println("Hystereis changed to: ");  
  Serial.print(hysteresisVar);  
});
```

Servo control

It is also possible to define a resource for controlling a servo instance. This way, the defined resource will automatically handle the servo instance, reading its current position, or changing to a new one according to the API interactions.

To define a servo resource, just define and initialize the servo as usual, and then use the declared instance in the resource definition.

```
thing["servo"] << servo(myServoInstance);
```

Communication between devices

In Thingier.io, it is possible that devices can communicate between them. There are two possibilities here. One is the communication between devices from the same account, and the other is the communication between devices from different accounts. Here we describe the two different approaches:

Same account communication

For this use case, in which both devices belong to the same user account, there is a specific method that allows devices to communicate with other devices with low latency and simple codification. This communication can contain data or not (it is possible to make an empty call). Let's suppose that we have two devices: `deviceA` and `deviceB`, and we want to communicate both calls from `deviceB` to a specific `deviceA` input resource. We can use `"thing.call_device(,);"`:

The `deviceA` defines a resource:

```
setup(){
  thing["resource_On_A"] = [](){
    Serial.println("Someone is calling me!");
  };
}
```

`deviceB` can easily call this resource and send data to it:

```
loop(){
  thing.handle();
  // be sure to call it at an appropriate rate
  thing.call_device("deviceA", "resource_On_A");
}
```

On the other hand, if we want to send the message with a `pson` payload in order to share data between devices. In this case, the `deviceA` will need to define a resource with some expected input

```

setup(){
    thing["resourceOnA"] << [](pson& in){
        int val1 = in["anyValue1"];
        float val2 = in["anyValue2"];
        // Work with the updated parameters here
    };
}

```

Then `deviceB` can call this method, providing the appropriate input by defining a `pson` type that is filled with the same keys used on `resourceOnA`:

```

loop(){
    thing.handle();
    // be sure to call it at an appropriate rate
    pson data;
    data["anyValue1"] = 3;
    data["anyValue2"] = 43.1;
    thing.call_device("deviceA", "resourceOnA", data);
}

```

`deviceB` can also call this method by providing the information from a defined resource that generates the information. In this case, the call is similar to the previous example, but using the resource as the data source.

```

setup(){
    thing["resourceName"] >> [](pson& out){
        out["anyValue1"] = 3;
        out["anyValue2"] = 43.1;
    };
}

loop(){
    thing.handle();
    // be sure to call it at an appropriate rate
    thing.call_device("deviceA", "resourceOnA",
thing["resourceName"]);
}

```

Communication between different accounts

If we want to communicate devices from different accounts, we can do that by calling an endpoint of type `Thingier.io Device Call`. Just register an endpoint of this type in the console:

Endpoint Details

Endpoint Identifier ⓘ DeviceACall

Endpoint Name ⓘ Endpoint

Endpoint Description ⓘ Endpoint for calling Device A from

Enabled ⓘ ☒

Endpoint Type ⓘ Thingier.io Device Call

Device Owner Username ⓘ otherUser

Device Identifier ⓘ deviceA

Resource Name ⓘ resourceOnA

Device Access Token ⓘ paste here a deviceA token device that gives you access to the device

✓ Add Endpoint

In this case, it is required to define different parameters in the endpoint:

- Endpoint Identifier: The endpoint ID that the device will use for calling the device.
- Endpoint Name: The name of the endpoint, which does not need to equal the "Endpoint Identifier". The endpoint will show in the list of endpoints with this name.
- Endpoint Description: This is an optional field. It is useful to remember what the endpoint consists of.
- Device Owner: The device owner's username.
- Device Identifier: The device ID of the other account.
- Resource Name: The resource on the device to be called.
- Device Access Token: A device token generated in the other account for granting external access to the device.

Once defined, the device will be able to call the endpoint, as explained in the following section. It basically consists of calling the `call_endpoint` method.

```
thing.call_endpoint("DeviceACall");
```

Using Endpoints

In Thingier.io, an endpoint is defined as some kind of external resource that can be accessed by the device. With the endpoints feature, devices can easily send emails, SMS, push data to external Web Services, interact with IFTTT, and perform any general action that can be made by using WebHooks (Calling HTTP/HTTPS URLs).

Calling an endpoint is so easy from the Arduino sketch, as it only requires calling the `call_endpoint` method over the `thing` variable.

```
thing.call_endpoint("endpoint_id");
```

Endpoints can be called from the device code in order to execute any action, like sending a predefined email. The call can also include some reading values, which is especially useful to send the device's data to third-party services.



Extra attention must be taken while calling resources, in order to avoid uncontrolled recurrency. If the interval is too short, the server will lock the device connection

Calling Endpoints

In this case, we will see a simple example to send an email alert based on a temperature value. For this example, we have configured an email endpoint `high_temp_email` that contains some warning text about the temperature. For this case, we do not want to check the temperature every millisecond, so we are introducing some variables to control the sensing and warning frequency. In this example, the temperature is checked every hour, and if it is above 30°C, it will call the endpoint called `high_temp_email` which will send us an email with the predefined text. It is important here **not to add delays** inside the loop method, as it will prevent the required execution of the `thing.handle()` method, so we are using a non-blocking delay based on the `millis()` function.

```

unsigned long lastCheck = 0;

loop(){
  thing.handle(); // required thing handle

  unsigned long currentTs = millis();

  if(currentTs-lastCheck>=60*60*1000){
    lastCheck = currentTs;
    if(dht.readTemperature(>30){
      thing.call_endpoint("high_temp_email");
    }
  }
}

```

Endpoints offer significant creative flexibility, allowing for automation based on various events. For example, endpoints can be triggered by a presence sensor detection, a humidity sensor reporting no water in plants, or a device's unexpected location. Furthermore, endpoints can be integrated with services like IFTTT (If This Then That) to interact with multiple third-party platforms.

Sending Data to Endpoints

Sending data to an endpoint (in JSON format) is also quite easy. We also need to call the `call_endpoint` method, but in this case, adding some information based on the `pson` data format, which will be automatically converted to JSON. For example, if we want to report data to a third-party service like Keen.io, we can create such kind of endpoint in the console. Once configured, we can call the endpoint with our readings, for example, with humidity and temperature values from a DHT sensor.

```

// be careful of sending data at an appropriate rate!
pson data;
data["temperature"] = dht.readTemperature();
data["humidity"] = dht.readHumidity();
thing.call_endpoint("keen_endpoint", data);

```

Data can also be sent based on a defined resource; for instance, if a resource already provides temperature and humidity. It is possible to reuse this definition for sending the same data to the endpoint, without having to redefine the sensor reading:

```

setup(){
  // defined resource in the setup for reading a sensor value
  thing["data"] >> (pson& out){
    out["temperature"] = dht.readTemperature();
    out["humidity"] = dht.readHumidity();
  }
}

loop(){
  // be careful of sending data at an appropriate rate!
  thing.call_endpoint("endpoint", thing["data"]);
}

```

Email Type Endpoint Example

This is a simple example, applied to an email-type endpoint, with a custom body

```

setup()
{
  thing["temperature"] >> outputValue(analogRead(0));
}
loop()
{
  if(actualLevel>UpperLevel && endpointUpperFlag)
  {
    thing.call_endpoint("endpoint_id",thing["temperature"]);
    endpointUpperFlag=0;
  }
}

```

Notice that there are a variable that limitates the run of this "if" just once, its important to define any condition or method to warrantee that this kind of enpoint call is executed just once (or at appropriate rate), because it can get a lot of emails generated by the microcontroller across thinger.io platform.

At endpoint configuration, in the custom body email, we must add double brackets "`{{<variable_key>}}`" to invoke the variable sent by the microcontroller.

```
"The room temperature is {{temperature}}%"
```


And receiving an email with the text:

```
The the room temperature is 80.34%
```

Using Data Buckets

Thingier.io provides an easy-to-use and extremely scalable virtual storage system that allows for storing long-term device data from device output resources. This information can be used to be plotted in dashboards, or can be exported in different formats for offline processing or a third-party Data Analysis process.

From Device Resource

It is not necessary to implement specific codification in device firmware to start storing data in a data bucket. Data buckets will retrieve information from output resources; simply configure the Data Bucket to set the source and sampling interval as explained in the [Console documentation](#). ↗

Streaming Resource Data

To enable the device to stream information only when required, such as upon event detection, the "Update by Device" option can be used during bucket configuration. This utilizes streaming resource instructions. For instance, using a previously defined Output Resource named "location," this could be achieved with this code snippet:

```
void loop() {
  thing.handle();
  // use the logic here to determine when to stream/record the
  resource.
  if(requires_recording){
    thing.stream("location");
  }
}
```

From Write Call

This option will allow setting the bucket in a state that it will not register any information by default, but it will just wait for writing calls, both from the Arduino library using the `write_bucket` method, as shown here, or calling the REST API directly, as done with Sigfox. This feature opens the option to register information in the same bucket from different devices, or store information from devices that are not connected permanently with the server, that are in sleep mode, or use a different technology like Sigfox.

Here is an example of an ESP8266 device writing information to a bucket using the `write_bucket` function:

```
void setup() {
  // define the resource with temperature and humidity
  thing["door_status"] >> [](pson &out){
    out["OPEN"] = (bool)digitalRead(SENSOR_PIN);
  };
}

void loop() {
  // handle connection
  thing.handle();

  if(digitalRead(SENSOR_PIN)!=previous_status){
    // write to bucket BucketId when the door changes its status
    thing.write_bucket("BucketId", "door_status");
  }
  previous_status=digitalRead(SENSOR_PIN);
}
```

Note that this instruction will retrieve the ["door_status"] resource PSON, so it is also possible to call this function by attaching a custom PSON:

```
void loop(){
  // handle connection
  thing.handle();

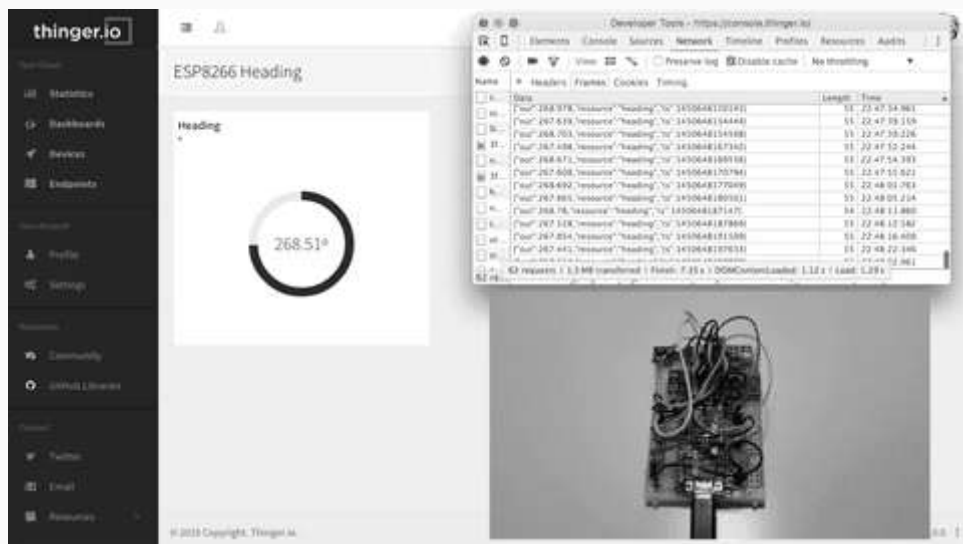
  if(digitalRead(SENSOR_PIN)!=previous_status){
    // write to bucket BucketId when the door changes its status
    thing.write_bucket("BucketId", "door_status");
  }
  previous_status=digitalRead(SENSOR_PIN);
}
```

Streaming Resources

In Thingier.io, WebSocket connections can be opened against devices to receive real-time sensor values, events, or other information. WebSockets are primarily utilized in the Console's Dashboard feature for streaming resources at a fixed, configurable interval. This functionality is available by default when an output resource is defined. However, to transmit information precisely when required, such as upon detection of movement or presence, a specific code, similar to calling an endpoint, must be programmed.

In such cases, it is necessary to detect when to stream the event, for example, when an accelerometer value exceeds a threshold, a presence sensor makes a detection, or the compass heading changes. The determination of when to stream new data is left to the implementer. Streaming resources also require that another endpoint is connected, listening for them (i.e., from a WebSocket connection), so if there is no one listening for this data, the data is not sent. This is handled automatically by the client library and the server, therefore, it is safe to stream data always, as the device will transmit the information only when there is a destination.

This example will report the compass heading in real-time if the heading value changes by more than 1 degree.



```
void setup(){
  thing["heading"] >> [](pson& out){
    out = getHeading();
  };
}

float previousHeading = 0;
void loop() {
  thing.handle();
  float currentHeading = getHeading();
  if(abs(currentHeading-previousHeading)>=1.0f){
    thing.stream(thing["heading"]);
    previousHeading=currentHeading;
  }
}
```

Enabling Debug Output

Thinger.io library provides extensive logging of its activities, which is especially useful when one needs to troubleshoot authentication and Wi-Fi connectivity issues. Include this definition in the sketch, but *make sure it comes first, before any other includes* (it was reported to cause crashes on some boards otherwise):

```
#define THINGER_SERIAL_DEBUG

// the rest of the sketch goes here
```

It is also necessary to enable `Serial` communication, as all the debugging information is displayed over the Serial. So, enable it in the sketch in the setup method.

```
void setup() {  
  Serial.begin(115200);  
}
```

Listen for Connection State

Sometimes it can be useful for an application to know the current connection status with Thingier.io, i.e., to notify disconnected status with a LED, request device configuration after authentication, or any other internal control flow according to connection state.

In order to create a listener for such connection states, it can be done with the `set_state_listener` function in the `setup()` method. For example, it is possible to define a listener that will receive the different connection states for the network, server, or authentication:

```

void setup(){

    // the setup code here..

    thing.set_state_listener([&](ThingierClient::THINGER_STATE
state){
        switch(state){
            case ThingierClient::NETWORK_CONNECTING:
                break;
            case ThingierClient::NETWORK_CONNECTED:
                break;
            case ThingierClient::NETWORK_CONNECT_ERROR:
                break;
            case ThingierClient::SOCKET_CONNECTING:
                break;
            case ThingierClient::SOCKET_CONNECTED:
                break;
            case ThingierClient::SOCKET_CONNECTION_ERROR:
                break;
            case ThingierClient::SOCKET_DISCONNECTED:
                break;
            case ThingierClient::SOCKET_ERROR:
                break;
            case ThingierClient::SOCKET_TIMEOUT:
                break;
            case ThingierClient::THINGER_AUTHENTICATING:
                break;
            case ThingierClient::THINGER_AUTHENTICATED:
                break;
            case ThingierClient::THINGER_AUTH_FAILED:
                break;
            case ThingierClient::THINGER_STOP_REQUEST:
                break;
        }
    });
}

```



In this table it is detailed the different values and their descriptions.

State	Description
NETWORK_CONNECTING	The underlying network is being connected, i.e., initializing ethernet, wifi, gsm, etc.
NETWORK_CONNECTED	The network is connected and ready to be used.
NETWORK_CONNECT_ERROR	The network cannot be initialized, i.e., bad WiFi credentials, cannot reach GSM, etc.
SOCKET_CONNECTING	After the network is connected, it means that the client is connecting to Thinger.io servers.
SOCKET_CONNECTED	The socket has been connected to the server.
SOCKET_CONNECTION_ERROR	The socket cannot be connected to Thinger.io. If often means a bad Internet connection.
SOCKET_DISCONNECTED	The connection with Thinger.io has been closed.
SOCKET_ERROR	An error happened with the socket, i.e, bad read or write, which will cause a disconnect.
SOCKET_TIMEOUT	The socket timed out while reading or writing, so the connection will be closed.
THINGER_AUTHENTICATING	Thinger.io client is connected and it is being authenticated.
THINGER_AUTHENTICATED	Thinger.io client is connected and authenticated, so it can use Thinger.io, i.e., call an endpoint, read a property, etc.
THINGER_AUTH_FAILED	Thinger.io client authentication failed. Please, review the server, username, device id, and password.
THINGER_STOP_REQUEST	Thinger.io client was requested to stop, i.e., from the source code, or by the server.

[Previous](#)
[OTHER DEVICES](#)

Next

TROUBLESHOOTING

Last updated 4 months ago

Was this helpful?

